

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

DATABASE MANAGEMENT SYSTEMS

LABORATORY MANUAL

Oracle SQL, PL/SQL & Database Design — Practical Record

Prepared by

V.Balu, Assistant Professor

CSE, SCSVMV UNIVERSITY

Academic Year 2026 – 2027

Index of Experiments

Ex. No.	Title of the Experiment
1	Creation of a Database and Writing SQL Queries to Retrieve Information from the Database (Oracle)
2	Performing Insertion, Deletion, Modifying, Altering, Updating and Viewing Records Based on Conditions (Oracle)
3	Practicing DCL and TCL Commands in Oracle
4	Creating a Database to Set Various Constraints in Oracle
5	Performing Queries Using ANY, ALL, IN, EXISTS, NOT EXISTS, UNION, INTERSECT and Constraints in Oracle
6	Queries Using Aggregate Functions, GROUP BY and HAVING in Oracle
7	Creation of Views, Sequence, Indexes and Savepoint in Oracle
8	Normalization in Oracle (1NF, 2NF, 3NF, 4NF, 5NF) Using Functional Dependencies
9	Creating Relationships Between Tables and Performing Joins and Subqueries in Oracle
10 (a)	PL/SQL Block to Satisfy Conditions by Accepting Input from the User
10 (b)	PL/SQL Block to Handle Exceptions in Oracle
10 (c)	Creation of Procedures and Functions in Oracle PL/SQL
10 (d)	Creation of Database Triggers (Insert Trigger, Delete Trigger, Update Trigger)
10 (e)	Creation of Database Operations with Implicit and Explicit Cursors in Oracle

Table of Contents

Index of Experiments	2
Table of Contents	3
Experiment No. 1	10
Aim	10
Software Required	10
Theory	10
Procedure.....	10
Program.....	10
Step 1: Create the STUDENT Table	10
Step 2: Insert Records into the STUDENT Table.....	10
Step 3: Display All Records.....	11
Step 4: Display Student Name and Marks.....	11
Step 5: Retrieve CSE Students.....	11
Step 6: Retrieve Students Scoring More Than 80 Marks	12
Viva Voce Questions	12
Result	12
Experiment No. 2	13
Aim	13
Software Required	13
Theory	13
Procedure.....	13
Program.....	13
Step 1: Create the EMPLOYEE Table	13
Step 2: Insert Records	14
Step 3: View All Records	14
Step 4: Update Records Based on a Condition	14
Step 5: Delete Records Based on a Condition.....	15
Step 6: Alter the Table Structure	15
Step 7: Modify a Column's Size	16
Step 8: View Records Based on Conditions.....	16
Viva Voce Questions	17
Result	17
Experiment No. 3	18

Aim	18
Software Required	18
Theory	18
Procedure.....	18
Program.....	18
Step 1: Create the EMPLOYEE Table	18
Step 2: Insert Records	18
DCL Commands	19
TCL Commands.....	19
Summary of Commands.....	21
Viva Voce Questions	21
Result	21
Experiment No. 4	22
Aim	22
Software Required	22
Theory	22
Procedure.....	22
Program.....	22
Step 1: Create the DEPARTMENT Table.....	22
Step 2: Create the EMPLOYEE Table with Constraints.....	23
Step 3: Insert Records into DEPARTMENT	23
Step 4: Insert Valid Records into EMPLOYEE	23
Testing the Constraints	23
Display Valid Records.....	24
Description of the Tables	25
Viva Voce Questions	25
Result	25
Experiment No. 5	26
Aim	26
Software Required	26
Theory	26
Procedure.....	26
Program.....	26
Step 1: Create the STUDENT Table	26
Step 2: Insert Records into STUDENT.....	27
Step 3: Create the ALUMNI Table	27

Step 4: Insert Records into ALUMNI	27
Query Using IN – Display CSE and IT Students	27
Query Using ANY – Students Scoring More Than ANY ECE Student	28
Query Using ALL – Students Scoring More Than ALL ECE Students	28
Query Using EXISTS – Students Who Are Also Alumni.....	28
Query Using NOT EXISTS – Students Not Found in Alumni.....	29
Query Using UNION – Combine Names from STUDENT and ALUMNI.....	29
Query Using INTERSECT – Display Common Names	29
Constraint Verification	30
Display All Records	30
Viva Voce Questions	30
Result	31
Experiment No. 6	32
Aim	32
Software Required	32
Theory	32
Procedure.....	32
Program.....	32
Step 1: Create the STUDENT Table	32
Step 2: Insert Records	33
Aggregate Function Queries	33
GROUP BY Clause	34
HAVING Clause	35
Display All Records	36
Viva Voce Questions	36
Result	36
Experiment No. 7	37
Aim	37
Software Required	37
Theory	37
Procedure.....	37
Program.....	37
Step 1: Create the STUDENT Table	37
Step 2: Insert Records	37
VIEW – Step 3: Create a View of CSE Students	38
Step 4: Retrieve Data from the View	38

SEQUENCE – Step 5: Create a Sequence	38
Step 6: Generate Sequence Values	38
Step 7: Insert Using the Sequence	39
INDEX – Step 8: Create an Index	39
SAVEPOINT – Step 9: Create a Savepoint.....	40
Step 10: Insert Another Record	40
Step 11: Roll Back to the Savepoint	40
Step 12: Verify the Records.....	40
Step 13: Commit the Transaction	41
Viva Voce Questions	41
Result	41
Experiment No. 8	42
Aim	42
Software Required	42
Theory	42
Procedure.....	42
Program.....	42
Unnormalized Form (UNF)	42
First Normal Form (1NF)	42
Second Normal Form (2NF).....	43
Third Normal Form (3NF).....	44
Fourth Normal Form (4NF).....	45
Fifth Normal Form (5NF)	46
Summary of Normal Forms	47
Viva Voce Questions	47
Result	48
Experiment No. 9	49
Aim	49
Software Required	49
Theory	49
Procedure.....	49
Program.....	49
Step 1: Create the DEPARTMENT Table.....	49
Step 2: Insert Records into DEPARTMENT	50
Step 3: Create the STUDENT Table	50
Step 4: Insert Records into STUDENT.....	50

INNER JOIN – Display Student and Department Details	50
LEFT OUTER JOIN – All Students with Department Information	51
RIGHT OUTER JOIN – All Departments and Students	51
FULL OUTER JOIN	51
SELF JOIN – Create the EMPLOYEE Table.....	52
SUBQUERY – Students Scoring Above Average Marks	52
SUBQUERY WITH IN – Students in the CSE Department	53
CORRELATED SUBQUERY – Students with Maximum Marks in Their Department	53
Viva Voce Questions	53
Result	54
Experiment No. 10 (a)	55
Aim	55
Software Required	55
Theory	55
Procedure.....	55
Program.....	55
PL/SQL Block	55
Sample Run 1 – Enter value for N: 25	56
Sample Run 2 – Enter value for N: -15.....	56
Sample Run 3 – Enter value for N: 0	56
Viva Voce Questions	56
Result	57
Experiment No. 10 (b).....	58
Aim	58
Software Required	58
Theory	58
Procedure.....	58
Program.....	58
Program 1: Handling the ZERO_DIVIDE Exception.....	58
Program 2: Handling the NO_DATA_FOUND Exception	59
Program 3: Handling the TOO_MANY_ROWS Exception.....	59
Program 4: Handling the DUP_VAL_ON_INDEX Exception	60
Program 5: Handling the VALUE_ERROR Exception	60
Program 6: Using the OTHERS Handler.....	61
Summary of Exceptions.....	61
Viva Voce Questions	61

Result	62
Experiment No. 10 (c)	63
Aim	63
Software Required	63
Theory	63
Procedure.....	63
Program.....	63
Step 1: Create the STUDENT Table	63
Step 2: Insert Sample Records	64
Step 3: Create a Procedure to Display Student Details.....	64
Step 4: Execute the Procedure.....	64
Step 5: Create a Function to Find the Grade	65
Step 6: Execute the Function	66
Step 7: Use the Function with Table Data.....	66
Verify the Created Objects.....	66
Viva Voce Questions	67
Result	67
Experiment No. 10 (d).....	68
Aim	68
Software Required	68
Theory	68
Procedure.....	68
Program.....	68
Step 1: Create the EMPLOYEE Table	68
Step 2: Create the EMP_AUDIT Table	69
Step 3: Insert Sample Records	69
INSERT Trigger – Step 4: Create the Trigger.....	69
UPDATE Trigger – Step 5: Create the Trigger.....	70
DELETE Trigger – Step 6: Create the Trigger	71
Display the EMPLOYEE Table	71
Explanation of :NEW and :OLD	72
Viva Voce Questions	72
Result	72
Experiment No. 10 (e).....	73
Aim	73
Software Required	73

Theory	73
Procedure.....	73
Program.....	73
Step 1: Create the EMPLOYEE Table	73
Step 2: Insert Sample Records	73
Part A – Implicit Cursor	74
Part B – Explicit Cursor.....	75
Using a Cursor FOR Loop.....	75
Implicit Cursor vs. Explicit Cursor.....	76
Viva Voce Questions	76
Result	77

Experiment No. 1

Creation of a Database and Writing SQL Queries to Retrieve Information from the Database (Oracle)

Aim

To create a table in an Oracle database, insert records into it, and retrieve information from the table using SQL queries.

Software Required

- Oracle Database 11g / 12c / 19c
- SQL*Plus or Oracle SQL Developer

Theory

SQL (Structured Query Language) is the standard language used to define, manipulate, and retrieve data from a relational database. Oracle organizes data into tables made up of rows and columns, and the SELECT statement is the primary tool used to retrieve information stored in those tables.

Procedure

1. Login to Oracle SQL*Plus or SQL Developer.
2. Create the required table.
3. Insert records into the table.
4. Execute SELECT queries to retrieve information.
5. Verify the output.

Program

Step 1: Create the STUDENT Table

```
CREATE TABLE STUDENT
(
    STUDENT_ID NUMBER(5) PRIMARY KEY,
    STUDENT_NAME VARCHAR2(30),
    DEPARTMENT VARCHAR2(20),
    AGE NUMBER(2),
    MARKS NUMBER(3)
);
```

Output

Table created.

Step 2: Insert Records into the STUDENT Table

```
INSERT INTO STUDENT VALUES (101,'ARUN','CSE',20,85);
INSERT INTO STUDENT VALUES (102,'BALA','ECE',21,78);
INSERT INTO STUDENT VALUES (103,'KAVI','CSE',20,92);
INSERT INTO STUDENT VALUES (104,'DIVYA','IT',21,88);
```

Output

```
1 row created.
1 row created.
1 row created.
1 row created.
```

Step 3: Display All Records

```
SELECT * FROM STUDENT;
```

Output

STUDENT_ID	STUDENT_NAME	DEPARTMENT	AGE	MARKS
-----	-----	-----	---	-----
101	ARUN	CSE	20	85
102	BALA	ECE	21	78
103	KAVI	CSE	20	92
104	DIVYA	IT	21	88

Step 4: Display Student Name and Marks

```
SELECT STUDENT_NAME, MARKS
FROM STUDENT;
```

Output

STUDENT_NAME	MARKS
-----	-----
ARUN	85
BALA	78
KAVI	92
DIVYA	88

Step 5: Retrieve CSE Students

```
SELECT *
FROM STUDENT
WHERE DEPARTMENT = 'CSE';
```

Output

STUDENT_ID	STUDENT_NAME	DEPARTMENT	AGE	MARKS
-----	-----	-----	---	-----
101	ARUN	CSE	20	85
103	KAVI	CSE	20	92

Step 6: Retrieve Students Scoring More Than 80 Marks

```
SELECT STUDENT_NAME, MARKS
FROM STUDENT
WHERE MARKS > 80;
```

Output

STUDENT_NAME	MARKS
ARUN	85
KAVI	92
DIVYA	88

Viva Voce Questions

1. What is SQL?
2. What is the purpose of the SELECT statement?
3. Differentiate between DDL and DML commands.
4. What is a primary key?
5. What is the use of the WHERE clause?
6. What are Oracle data types?
7. What is the purpose of the INSERT command?
8. How do you retrieve all records from a table?
9. What is the syntax of CREATE TABLE?
10. What is a database?

Result

Thus, the STUDENT table was created successfully, records were inserted, and SQL queries were executed to retrieve information from the Oracle database successfully.

Experiment No. 2

Performing Insertion, Deletion, Modifying, Altering, Updating and Viewing Records Based on Conditions (Oracle)

Aim

To perform INSERT, DELETE, MODIFY, ALTER, UPDATE, and SELECT operations on a table using Oracle SQL.

Software Required

- Oracle Database 11g / 12c / 19c
- SQL*Plus or Oracle SQL Developer

Theory

Data Manipulation Language (DML) commands are used to manipulate the data stored within database tables, while Data Definition Language (DDL) commands are used to modify the structure of database objects themselves.

Command	Purpose
INSERT	Adds new records to a table
DELETE	Removes existing records from a table
UPDATE	Modifies values in existing records
ALTER	Changes the structure of a table
SELECT	Retrieves records based on specified conditions

Procedure

1. Create a table named EMPLOYEE.
2. Insert records into the table.
3. View all records.
4. Update records based on conditions.
5. Delete records based on conditions.
6. Alter the table structure.
7. Retrieve records using the WHERE clause.

Program

Step 1: Create the EMPLOYEE Table

```
CREATE TABLE EMPLOYEE  
(
```

```
EMP_ID NUMBER(5) PRIMARY KEY,  
EMP_NAME VARCHAR2(30),  
DEPARTMENT VARCHAR2(20),  
SALARY NUMBER(8),  
CITY VARCHAR2(20)  
);
```

Output

Table created.

Step 2: Insert Records

```
INSERT INTO EMPLOYEE VALUES (101, 'ARUN', 'CSE', 35000, 'CHENNAI');  
INSERT INTO EMPLOYEE VALUES (102, 'BALA', 'ECE', 32000, 'VELLORE');  
INSERT INTO EMPLOYEE VALUES (103, 'DIVYA', 'IT', 40000, 'KANCHIPURAM');  
INSERT INTO EMPLOYEE VALUES (104, 'KAVI', 'CSE', 38000, 'CHENNAI');
```

Output

1 row created.
1 row created.
1 row created.
1 row created.

Step 3: View All Records

```
SELECT * FROM EMPLOYEE;
```

Output

EMP_ID	EMP_NAME	DEPARTMENT	SALARY	CITY
-----	-----	-----	-----	-----
101	ARUN	CSE	35000	CHENNAI
102	BALA	ECE	32000	VELLORE
103	DIVYA	IT	40000	KANCHIPURAM
104	KAVI	CSE	38000	CHENNAI

Step 4: Update Records Based on a Condition

Increase the salary of the employee with EMP_ID 102.

```
UPDATE EMPLOYEE  
SET SALARY = 35000  
WHERE EMP_ID = 102;
```

Output

1 row updated.

Verify the update:

```
SELECT * FROM EMPLOYEE
WHERE EMP_ID = 102;
```

Output

EMP_ID	EMP_NAME	DEPARTMENT	SALARY	CITY
102	BALA	ECE	35000	VELLORE

Step 5: Delete Records Based on a Condition

Delete the employee belonging to the IT department.

```
DELETE FROM EMPLOYEE
WHERE DEPARTMENT = 'IT';
```

Output

1 row deleted.

Verify the deletion:

```
SELECT * FROM EMPLOYEE;
```

Output

EMP_ID	EMP_NAME	DEPARTMENT	SALARY	CITY
101	ARUN	CSE	35000	CHENNAI
102	BALA	ECE	35000	VELLORE
104	KAVI	CSE	38000	CHENNAI

Step 6: Alter the Table Structure

Add a new column:

```
ALTER TABLE EMPLOYEE
ADD EMAIL VARCHAR2(40);
```

Output

Table altered.

Describe the table:

```
DESC EMPLOYEE;
```

Output

Name	Null?	Type
EMP_ID	NOT NULL	NUMBER(5)
EMP_NAME		VARCHAR2(30)
DEPARTMENT		VARCHAR2(20)

SALARY	NUMBER(8)
CITY	VARCHAR2(20)
EMAIL	VARCHAR2(40)

Step 7: Modify a Column's Size

```
ALTER TABLE EMPLOYEE
MODIFY EMP_NAME VARCHAR2(50);
```

Output

Table altered.

Step 8: View Records Based on Conditions

Employees working in the CSE department:

```
SELECT *
FROM EMPLOYEE
WHERE DEPARTMENT = 'CSE';
```

Output

EMP_ID	EMP_NAME	DEPARTMENT	SALARY	CITY
-----	-----	-----	-----	-----
101	ARUN	CSE	35000	CHENNAI
104	KAVI	CSE	38000	CHENNAI

Employees with salary greater than 35,000 (evaluated before deletion of DIVYA):

```
SELECT EMP_NAME, SALARY
FROM EMPLOYEE
WHERE SALARY > 35000;
```

Output

EMP_NAME	SALARY
-----	-----
DIVYA	40000
KAVI	38000

Employees based in Chennai:

```
SELECT EMP_NAME, CITY
FROM EMPLOYEE
WHERE CITY = 'CHENNAI';
```

Output

EMP_NAME	CITY
-----	-----
ARUN	CHENNAI
KAVI	CHENNAI

Viva Voce Questions

1. What is the purpose of the INSERT command?
2. Differentiate between DELETE and TRUNCATE.
3. What is the use of the UPDATE statement?
4. Explain the ALTER TABLE command.
5. What is the WHERE clause?
6. What happens if the WHERE clause is omitted in a DELETE statement?
7. What is DML?
8. What is DDL?
9. How do you add a new column to a table?
10. How do you modify an existing column?

Result

Thus, the INSERT, DELETE, MODIFY, ALTER, UPDATE, and SELECT operations were performed successfully in Oracle, and records were viewed based on the specified conditions.

Experiment No. 3

Practicing DCL and TCL Commands in Oracle

Aim

To practice Data Control Language (DCL) and Transaction Control Language (TCL) commands in an Oracle database.

Software Required

- Oracle Database 11g / 12c / 19c
- SQL*Plus or Oracle SQL Developer

Theory

Data Control Language (DCL) commands are used to control access privileges to database objects. GRANT provides privileges to users, while REVOKE removes privileges that were previously granted.

Transaction Control Language (TCL) commands are used to manage transactions within a database. COMMIT permanently saves changes made during the current transaction, ROLLBACK undoes changes made during the current transaction, and SAVEPOINT creates a marked point within a transaction to which a rollback can later be performed.

Procedure

1. Create a table.
2. Insert records into the table.
3. Grant privileges to another user.
4. Revoke privileges from the user.
5. Practice COMMIT, SAVEPOINT, and ROLLBACK commands.
6. Verify the results.

Program

Step 1: Create the EMPLOYEE Table

```
CREATE TABLE EMPLOYEE
(
    EMP_ID NUMBER(5) PRIMARY KEY,
    EMP_NAME VARCHAR2(30),
    SALARY NUMBER(8)
);
```

Output

Table created.

Step 2: Insert Records

```
INSERT INTO EMPLOYEE VALUES (101, 'ARUN', 35000);  
INSERT INTO EMPLOYEE VALUES (102, 'BALA', 40000);
```

Output

```
1 row created.  
1 row created.
```

DCL Commands

Step 3: Grant the SELECT privilege on the EMPLOYEE table to user SCOTT.

```
GRANT SELECT  
ON EMPLOYEE  
TO SCOTT;
```

Output

```
Grant succeeded.
```

Step 4: Grant multiple privileges.

```
GRANT SELECT, INSERT, UPDATE  
ON EMPLOYEE  
TO SCOTT;
```

Output

```
Grant succeeded.
```

Step 5: Remove the INSERT privilege from SCOTT.

```
REVOKE INSERT  
ON EMPLOYEE  
FROM SCOTT;
```

Output

```
Revoke succeeded.
```

TCL Commands

Step 6: Insert a new record and commit the transaction.

```
INSERT INTO EMPLOYEE VALUES (103, 'DIVYA', 45000);
```

Output

```
1 row created.
```

```
COMMIT;
```

Output

```
Commit complete.
```

Step 7: Insert another record and create a savepoint.

```
INSERT INTO EMPLOYEE VALUES (104, 'KAVI', 42000);
```

Output

```
1 row created.
```

```
SAVEPOINT SP1;
```

Output

```
Savepoint created.
```

Step 8: Insert an additional record.

```
INSERT INTO EMPLOYEE VALUES (105, 'RAVI', 38000);
```

Output

```
1 row created.
```

Step 9: Roll back to the savepoint.

```
ROLLBACK TO SP1;
```

Output

```
Rollback complete.
```

Step 10: Verify the records.

```
SELECT * FROM EMPLOYEE;
```

Output

EMP_ID	EMP_NAME	SALARY
-----	-----	-----
101	ARUN	35000
102	BALA	40000
103	DIVYA	45000
104	KAVI	42000

Record 105 (RAVI) is removed because of the rollback to savepoint SP1.

Step 11: Complete the transaction.

```
COMMIT;
```

Output

```
Commit complete.
```

Summary of Commands

Command	Purpose
GRANT	Gives privileges to users
REVOKE	Removes privileges from users
COMMIT	Saves all changes permanently
SAVEPOINT	Creates a rollback point within a transaction
ROLLBACK	Undoes changes made since a given point

Viva Voce Questions

1. What is DCL?
2. What is TCL?
3. What is the purpose of the GRANT command?
4. What is the syntax of REVOKE?
5. What is COMMIT?
6. What is ROLLBACK?
7. What is a SAVEPOINT?
8. Can a committed transaction be rolled back?
9. Differentiate between DCL and TCL commands.
10. What happens if ROLLBACK is issued without a SAVEPOINT?

Result

Thus, the DCL commands (GRANT and REVOKE) and the TCL commands (COMMIT, SAVEPOINT, and ROLLBACK) were executed successfully in an Oracle database.

Experiment No. 4

Creating a Database to Set Various Constraints in Oracle

Aim

To create database tables and implement various constraints such as PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, and CHECK in Oracle.

Software Required

- Oracle Database 11g / 12c / 19c
- SQL*Plus or Oracle SQL Developer

Theory

Constraints are rules enforced on data columns to maintain the accuracy, integrity, and consistency of data stored in a database.

Constraint	Description
NOT NULL	Ensures that a column cannot contain NULL values
UNIQUE	Ensures that all values in a column are distinct
PRIMARY KEY	Uniquely identifies each record in a table
FOREIGN KEY	Establishes a relationship between two tables
CHECK	Restricts the range of values that can be entered into a column

Procedure

1. Create a DEPARTMENT table with a primary key.
2. Create an EMPLOYEE table using various constraints.
3. Insert valid records.
4. Attempt to insert invalid records.
5. Observe the constraint violations.
6. Display the records.

Program

Step 1: Create the DEPARTMENT Table

```
CREATE TABLE DEPARTMENT
(
    DEPT_ID NUMBER(3),
    DEPT_NAME VARCHAR2(30),

    CONSTRAINT PK_DEPT
```

```
PRIMARY KEY (DEPT_ID)
);
```

Output

Table created.

Step 2: Create the EMPLOYEE Table with Constraints

```
CREATE TABLE EMPLOYEE
(
  EMP_ID NUMBER(5),
  EMP_NAME VARCHAR2(30) CONSTRAINT NN_EMPNAME NOT NULL,
  EMAIL VARCHAR2(50),
  SALARY NUMBER(8),
  DEPT_ID NUMBER(3),

  CONSTRAINT PK_EMP PRIMARY KEY (EMP_ID),
  CONSTRAINT UK_EMAIL UNIQUE (EMAIL),
  CONSTRAINT CK_SALARY CHECK (SALARY >= 10000),
  CONSTRAINT FK_DEPT FOREIGN KEY (DEPT_ID)
    REFERENCES DEPARTMENT(DEPT_ID)
);
```

Output

Table created.

Step 3: Insert Records into DEPARTMENT

```
INSERT INTO DEPARTMENT VALUES (101,'CSE');
INSERT INTO DEPARTMENT VALUES (102,'ECE');
```

Output

1 row created.
1 row created.

Step 4: Insert Valid Records into EMPLOYEE

```
INSERT INTO EMPLOYEE VALUES (1001,'ARUN','arun@gmail.com',35000,101);
INSERT INTO EMPLOYEE VALUES (1002,'BALA','bala@gmail.com',30000,102);
```

Output

1 row created.
1 row created.

Testing the Constraints

NOT NULL – attempt to insert a NULL value into EMP_NAME:

```
INSERT INTO EMPLOYEE  
VALUES (1003,NULL,'divya@gmail.com',25000,101);
```

Output

```
ERROR at line 1:  
ORA-01400: cannot insert NULL into ("EMPLOYEE"."EMP_NAME")
```

UNIQUE – attempt to insert a duplicate email:

```
INSERT INTO EMPLOYEE  
VALUES (1004,'KAVI','arun@gmail.com',28000,101);
```

Output

```
ERROR at line 1:  
ORA-00001: unique constraint (UK_EMAIL) violated
```

CHECK – attempt to insert a salary less than 10000:

```
INSERT INTO EMPLOYEE  
VALUES (1005,'RAVI','ravi@gmail.com',8000,101);
```

Output

```
ERROR at line 1:  
ORA-02290: check constraint (CK_SALARY) violated
```

FOREIGN KEY – attempt to insert an invalid department:

```
INSERT INTO EMPLOYEE  
VALUES (1006,'SURESH','suresh@gmail.com',25000,105);
```

Output

```
ERROR at line 1:  
ORA-02291: integrity constraint (FK_DEPT) violated  
parent key not found
```

PRIMARY KEY – attempt to insert a duplicate EMP_ID:

```
INSERT INTO EMPLOYEE  
VALUES (1001,'MANOJ','manoj@gmail.com',30000,101);
```

Output

```
ERROR at line 1:  
ORA-00001: unique constraint (PK_EMP) violated
```

Display Valid Records

```
SELECT * FROM EMPLOYEE;
```

Output

EMP_ID	EMP_NAME	EMAIL	SALARY	DEPT_ID
1001	ARUN	arun@gmail.com	35000	101
1002	BALA	bala@gmail.com	30000	102

Description of the Tables

```
DESC DEPARTMENT;
```

Output

Name	Null?	Type
DEPT_ID	NOT NULL	NUMBER(3)
DEPT_NAME		VARCHAR2(30)

```
DESC EMPLOYEE;
```

Output

Name	Null?	Type
EMP_ID	NOT NULL	NUMBER(5)
EMP_NAME	NOT NULL	VARCHAR2(30)
EMAIL		VARCHAR2(50)
SALARY		NUMBER(8)
DEPT_ID		NUMBER(3)

Viva Voce Questions

1. What is a constraint?
2. Why are constraints used in databases?
3. Explain the PRIMARY KEY constraint.
4. What is the difference between PRIMARY KEY and UNIQUE?
5. What is a FOREIGN KEY?
6. What is the purpose of the CHECK constraint?
7. Can a PRIMARY KEY contain NULL values?
8. What is the use of the NOT NULL constraint?
9. What error occurs when a foreign key reference is invalid?
10. How many PRIMARY KEY constraints can a table have?

Result

Thus, the DEPARTMENT and EMPLOYEE tables were created successfully, and the PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, and CHECK constraints were implemented and verified in an Oracle database.

Experiment No. 5

Performing Queries Using ANY, ALL, IN, EXISTS, NOT EXISTS, UNION, INTERSECT and Constraints in Oracle

Aim

To perform SQL queries using the ANY, ALL, IN, EXISTS, NOT EXISTS, UNION, and INTERSECT operators, and to verify constraints in an Oracle database.

Software Required

- Oracle Database 11g / 12c / 19c
- SQL*Plus or Oracle SQL Developer

Theory

Oracle provides several advanced SQL operators that allow data to be retrieved and combined efficiently.

Operator	Description
IN	Checks whether a value matches any value in a list
ANY	Compares a value with any value returned by a subquery
ALL	Compares a value with all values returned by a subquery
EXISTS	Returns TRUE if the subquery returns at least one row
NOT EXISTS	Returns TRUE if the subquery returns no rows
UNION	Combines the results of two queries and removes duplicates
INTERSECT	Returns only the records common to two queries

Procedure

1. Create the STUDENT and ALUMNI tables.
2. Insert records into the tables.
3. Execute queries using IN, ANY, and ALL.
4. Execute EXISTS and NOT EXISTS queries.
5. Perform UNION and INTERSECT operations.
6. Verify the output.

Program

Step 1: Create the STUDENT Table

```
CREATE TABLE STUDENT
```

```
(  
    STUDENT_ID NUMBER(5) PRIMARY KEY,  
    STUDENT_NAME VARCHAR2(30),  
    DEPARTMENT VARCHAR2(20),  
    MARKS NUMBER(3)  
);
```

Output

Table created.

Step 2: Insert Records into STUDENT

```
INSERT INTO STUDENT VALUES (101,'ARUN','CSE',85);  
INSERT INTO STUDENT VALUES (102,'BALA','ECE',78);  
INSERT INTO STUDENT VALUES (103,'DIVYA','IT',92);  
INSERT INTO STUDENT VALUES (104,'KAVI','CSE',88);
```

Output

1 row created.
1 row created.
1 row created.
1 row created.

Step 3: Create the ALUMNI Table

```
CREATE TABLE ALUMNI  
(  
    STUDENT_NAME VARCHAR2(30)  
);
```

Output

Table created.

Step 4: Insert Records into ALUMNI

```
INSERT INTO ALUMNI VALUES ('ARUN');  
INSERT INTO ALUMNI VALUES ('RAVI');  
INSERT INTO ALUMNI VALUES ('KAVI');
```

Output

1 row created.
1 row created.
1 row created.

Query Using IN – Display CSE and IT Students

```
SELECT *
```

```
FROM STUDENT
WHERE DEPARTMENT IN ('CSE','IT');
```

Output

STUDENT_ID	STUDENT_NAME	DEPARTMENT	MARKS
-----	-----	-----	-----
101	ARUN	CSE	85
103	DIVYA	IT	92
104	KAVI	CSE	88

Query Using ANY – Students Scoring More Than ANY ECE Student

```
SELECT *
FROM STUDENT
WHERE MARKS > ANY
(
    SELECT MARKS
    FROM STUDENT
    WHERE DEPARTMENT = 'ECE'
);
```

Output

STUDENT_ID	STUDENT_NAME	DEPARTMENT	MARKS
-----	-----	-----	-----
101	ARUN	CSE	85
103	DIVYA	IT	92
104	KAVI	CSE	88

Query Using ALL – Students Scoring More Than ALL ECE Students

```
SELECT *
FROM STUDENT
WHERE MARKS > ALL
(
    SELECT MARKS
    FROM STUDENT
    WHERE DEPARTMENT = 'ECE'
);
```

Output

STUDENT_ID	STUDENT_NAME	DEPARTMENT	MARKS
-----	-----	-----	-----
101	ARUN	CSE	85
103	DIVYA	IT	92
104	KAVI	CSE	88

Query Using EXISTS – Students Who Are Also Alumni

```
SELECT STUDENT_NAME
FROM STUDENT S
WHERE EXISTS
(
    SELECT *
    FROM ALUMNI A
    WHERE A.STUDENT_NAME = S.STUDENT_NAME
);
```

Output

```
STUDENT_NAME
-----
ARUN
KAVI
```

Query Using NOT EXISTS – Students Not Found in Alumni

```
SELECT STUDENT_NAME
FROM STUDENT S
WHERE NOT EXISTS
(
    SELECT *
    FROM ALUMNI A
    WHERE A.STUDENT_NAME = S.STUDENT_NAME
);
```

Output

```
STUDENT_NAME
-----
BALA
DIVYA
```

Query Using UNION – Combine Names from STUDENT and ALUMNI

```
SELECT STUDENT_NAME FROM STUDENT
UNION
SELECT STUDENT_NAME FROM ALUMNI;
```

Output

```
STUDENT_NAME
-----
ARUN
BALA
DIVYA
KAVI
RAVI
```

Query Using INTERSECT – Display Common Names

```
SELECT STUDENT_NAME FROM STUDENT
INTERSECT
SELECT STUDENT_NAME FROM ALUMNI;
```

Output

```
STUDENT_NAME
-----
ARUN
KAVI
```

Constraint Verification

Attempt to insert a duplicate primary key:

```
INSERT INTO STUDENT VALUES (101,'SURESH','EEE',80);
```

Output

```
ERROR at line 1:
ORA-00001: unique constraint violated
```

Attempt to insert a NULL primary key:

```
INSERT INTO STUDENT VALUES (NULL,'MANOJ','CSE',75);
```

Output

```
ERROR at line 1:
ORA-01400: cannot insert NULL into ("STUDENT"."STUDENT_ID")
```

Display All Records

```
SELECT * FROM STUDENT;
```

Output

STUDENT_ID	STUDENT_NAME	DEPARTMENT	MARKS
-----	-----	-----	-----
101	ARUN	CSE	85
102	BALA	ECE	78
103	DIVYA	IT	92
104	KAVI	CSE	88

Viva Voce Questions

1. What is the difference between ANY and ALL?
2. Explain the IN operator.
3. What is EXISTS?
4. Differentiate between EXISTS and NOT EXISTS.
5. What is UNION?

6. How does INTERSECT work?
7. What are constraints?
8. What is a PRIMARY KEY?
9. Why are subqueries used?
10. What is the purpose of advanced SQL operators?

Result

Thus, SQL queries using ANY, ALL, IN, EXISTS, NOT EXISTS, UNION, and INTERSECT, along with constraint verification, were executed successfully and the corresponding outputs were verified in an Oracle database.

Experiment No. 6

Queries Using Aggregate Functions, GROUP BY and HAVING in Oracle

Aim

To perform SQL queries using aggregate functions, the GROUP BY clause, and the HAVING clause in an Oracle database.

Software Required

- Oracle Database 11g / 12c / 19c
- SQL*Plus or Oracle SQL Developer

Theory

Aggregate functions perform a calculation on a set of values and return a single summary value.

Function	Description
COUNT()	Counts the number of rows
SUM()	Calculates the total of a set of values
AVG()	Calculates the average value
MAX()	Finds the maximum value
MIN()	Finds the minimum value

The GROUP BY clause groups rows that share the same values into summary rows, while the HAVING clause is used to filter the results of those grouped records.

Procedure

1. Create the STUDENT table.
2. Insert records into the table.
3. Execute aggregate function queries.
4. Use GROUP BY to categorize records.
5. Apply HAVING conditions on grouped data.
6. Verify the results.

Program

Step 1: Create the STUDENT Table

```
CREATE TABLE STUDENT
(
  STUDENT_ID NUMBER(5) PRIMARY KEY,
  STUDENT_NAME VARCHAR2(30),
  DEPARTMENT VARCHAR2(20),
```

```
MARKS NUMBER(3)
);
```

Output

Table created.

Step 2: Insert Records

```
INSERT INTO STUDENT VALUES (101,'ARUN','CSE',85);
INSERT INTO STUDENT VALUES (102,'BALA','ECE',78);
INSERT INTO STUDENT VALUES (103,'DIVYA','IT',92);
INSERT INTO STUDENT VALUES (104,'KAVI','CSE',88);
INSERT INTO STUDENT VALUES (105,'RAVI','ECE',81);
INSERT INTO STUDENT VALUES (106,'SURESH','IT',95);
```

Output

```
1 row created.
1 row created.
1 row created.
1 row created.
1 row created.
1 row created.
```

Aggregate Function Queries

Count the total number of students:

```
SELECT COUNT(*) AS TOTAL_STUDENTS
FROM STUDENT;
```

Output

```
TOTAL_STUDENTS
-----
6
```

Calculate total marks:

```
SELECT SUM(MARKS) AS TOTAL_MARKS
FROM STUDENT;
```

Output

```
TOTAL_MARKS
-----
519
```

Calculate average marks:

```
SELECT AVG(MARKS) AS AVERAGE_MARKS
FROM STUDENT;
```

Output

```
AVERAGE_MARKS
-----
86.5
```

Find the highest mark:

```
SELECT MAX(MARKS) AS HIGHEST_MARK
FROM STUDENT;
```

Output

```
HIGHEST_MARK
-----
95
```

Find the lowest mark:

```
SELECT MIN(MARKS) AS LOWEST_MARK
FROM STUDENT;
```

Output

```
LOWEST_MARK
-----
78
```

GROUP BY Clause*Average marks, department-wise:*

```
SELECT DEPARTMENT, AVG(MARKS) AS AVG_MARKS
FROM STUDENT
GROUP BY DEPARTMENT;
```

Output

DEPARTMENT	AVG_MARKS
-----	-----
CSE	86.5
ECE	79.5
IT	93.5

Count students in each department:

```
SELECT DEPARTMENT, COUNT(*) AS TOTAL_STUDENTS
FROM STUDENT
GROUP BY DEPARTMENT;
```

Output

DEPARTMENT	TOTAL_STUDENTS
-----	-----
CSE	2

ECE	2
IT	2

Department-wise maximum marks:

```
SELECT DEPARTMENT, MAX(MARKS) AS MAX_MARKS
FROM STUDENT
GROUP BY DEPARTMENT;
```

Output

DEPARTMENT	MAX_MARKS
-----	-----
CSE	88
ECE	81
IT	95

HAVING Clause

Departments with average marks greater than 85:

```
SELECT DEPARTMENT, AVG(MARKS) AS AVG_MARKS
FROM STUDENT
GROUP BY DEPARTMENT
HAVING AVG(MARKS) > 85;
```

Output

DEPARTMENT	AVG_MARKS
-----	-----
CSE	86.5
IT	93.5

Departments having more than one student:

```
SELECT DEPARTMENT, COUNT(*) AS TOTAL_STUDENTS
FROM STUDENT
GROUP BY DEPARTMENT
HAVING COUNT(*) > 1;
```

Output

DEPARTMENT	TOTAL_STUDENTS
-----	-----
CSE	2
ECE	2
IT	2

Combined query using GROUP BY and HAVING:

```
SELECT DEPARTMENT, COUNT(*) AS NO_OF_STUDENTS,
      AVG(MARKS) AS AVG_MARKS, MAX(MARKS) AS HIGHEST_MARK
FROM STUDENT
GROUP BY DEPARTMENT
```

```
HAVING AVG(MARKS) > 80;
```

Output

DEPARTMENT	NO_OF_STUDENTS	AVG_MARKS	HIGHEST_MARK
-----	-----	-----	-----
CSE	2	86.5	88
IT	2	93.5	95

Display All Records

```
SELECT * FROM STUDENT;
```

Output

STUDENT_ID	STUDENT_NAME	DEPARTMENT	MARKS
-----	-----	-----	-----
101	ARUN	CSE	85
102	BALA	ECE	78
103	DIVYA	IT	92
104	KAVI	CSE	88
105	RAVI	ECE	81
106	SURESH	IT	95

Viva Voce Questions

1. What are aggregate functions?
2. Explain the COUNT() function.
3. What is the difference between SUM() and AVG()?
4. What is the purpose of GROUP BY?
5. Why is the HAVING clause used?
6. Differentiate between the WHERE and HAVING clauses.
7. Can HAVING be used without GROUP BY?
8. What is the use of MAX() and MIN()?
9. What happens if GROUP BY is omitted?
10. Explain the syntax of aggregate functions.

Result

Thus, SQL queries using aggregate functions, GROUP BY, and HAVING clauses were executed successfully, and the outputs were verified in an Oracle database.

Experiment No. 7

Creation of Views, Sequence, Indexes and Savepoint in Oracle

Aim

To create and use Views, Sequences, Indexes, and Savepoints in an Oracle database.

Software Required

- Oracle Database 11g / 12c / 19c
- SQL*Plus or Oracle SQL Developer

Theory

A View is a virtual table built from one or more underlying tables; it does not store data physically, but instead displays data drawn from those tables. A Sequence is a database object used to generate unique numbers automatically. An Index improves the speed of data retrieval operations on a table. A Savepoint is a marked point within a transaction to which a rollback can later be performed.

Procedure

1. Create a STUDENT table.
2. Insert records into the table.
3. Create and retrieve data using a View.
4. Create a Sequence and generate sequence values.
5. Create an Index on a table column.
6. Use Savepoint and Rollback.
7. Verify the outputs.

Program

Step 1: Create the STUDENT Table

```
CREATE TABLE STUDENT
(
  STUDENT_ID NUMBER(5) PRIMARY KEY,
  STUDENT_NAME VARCHAR2(30),
  DEPARTMENT VARCHAR2(20),
  MARKS NUMBER(3)
);
```

Output

Table created.

Step 2: Insert Records

```
INSERT INTO STUDENT VALUES (101,'ARUN','CSE',85);
INSERT INTO STUDENT VALUES (102,'BALA','ECE',78);
INSERT INTO STUDENT VALUES (103,'DIVYA','IT',92);
INSERT INTO STUDENT VALUES (104,'KAVI','CSE',88);
```

Output

```
1 row created.
1 row created.
1 row created.
1 row created.
```

VIEW – Step 3: Create a View of CSE Students

```
CREATE VIEW CSE_STUDENTS AS
SELECT STUDENT_ID, STUDENT_NAME, MARKS
FROM STUDENT
WHERE DEPARTMENT = 'CSE';
```

Output

```
View created.
```

Step 4: Retrieve Data from the View

```
SELECT * FROM CSE_STUDENTS;
```

Output

STUDENT_ID	STUDENT_NAME	MARKS
-----	-----	-----
101	ARUN	85
104	KAVI	88

SEQUENCE – Step 5: Create a Sequence

```
CREATE SEQUENCE STUD_SEQ
START WITH 105
INCREMENT BY 1
MINVALUE 105
MAXVALUE 999
NOCYCLE
NOCACHE;
```

Output

```
Sequence created.
```

Step 6: Generate Sequence Values

```
SELECT STUD_SEQ.NEXTVAL FROM DUAL;
```

Output

```
NEXTVAL
-----
105
```

Executed again:

```
SELECT STUD_SEQ.NEXTVAL FROM DUAL;
```

Output

```
NEXTVAL
-----
106
```

Step 7: Insert Using the Sequence

```
INSERT INTO STUDENT
VALUES (STUD_SEQ.NEXTVAL, 'RAVI', 'ECE', 81);
```

Output

```
1 row created.
```

Verify the insert:

```
SELECT * FROM STUDENT;
```

Output

STUDENT_ID	STUDENT_NAME	DEPARTMENT	MARKS
-----	-----	-----	-----
101	ARUN	CSE	85
102	BALA	ECE	78
103	DIVYA	IT	92
104	KAVI	CSE	88
107	RAVI	ECE	81

INDEX – Step 8: Create an Index

```
CREATE INDEX IDX_STUDENT_NAME
ON STUDENT(STUDENT_NAME);
```

Output

```
Index created.
```

Verify the index:

```
SELECT INDEX_NAME, TABLE_NAME
FROM USER_INDEXES
WHERE TABLE_NAME = 'STUDENT';
```

Output

INDEX_NAME	TABLE_NAME
-----	-----
IDX_STUDENT_NAME	STUDENT
SYS_C001234	STUDENT

SYS_C001234 is the index automatically generated for the primary key.

SAVEPOINT – Step 9: Create a Savepoint

```
INSERT INTO STUDENT VALUES (108,'SURESH','IT',90);
```

Output

1 row created.

```
SAVEPOINT SP1;
```

Output

Savepoint created.

Step 10: Insert Another Record

```
INSERT INTO STUDENT VALUES (109,'MANOJ','CSE',87);
```

Output

1 row created.

Step 11: Roll Back to the Savepoint

```
ROLLBACK TO SP1;
```

Output

Rollback complete.

Step 12: Verify the Records

```
SELECT * FROM STUDENT;
```

Output

STUDENT_ID	STUDENT_NAME	DEPARTMENT	MARKS
-----	-----	-----	-----
101	ARUN	CSE	85
102	BALA	ECE	78
103	DIVYA	IT	92
104	KAVI	CSE	88
107	RAVI	ECE	81
108	SURESH	IT	90

Record 109 (MANOJ) is removed because of the rollback to savepoint SP1.

Step 13: Commit the Transaction

```
COMMIT;
```

Output

```
Commit complete.
```

Viva Voce Questions

1. What is a View?
2. What are the advantages of Views?
3. What is a Sequence?
4. Explain NEXTVAL and CURRVAL.
5. What is an Index?
6. Why are Indexes used?
7. What is a Savepoint?
8. Differentiate between COMMIT and ROLLBACK.
9. Can a View store data physically?
10. What happens when ROLLBACK TO SAVEPOINT is executed?

Result

Thus, Views, Sequences, Indexes, and Savepoints were created and used successfully, and the outputs were verified in an Oracle database.

Experiment No. 8

Normalization in Oracle (1NF, 2NF, 3NF, 4NF, 5NF) Using Functional Dependencies

Aim

To study and perform normalization up to the Fifth Normal Form (5NF) using functional dependencies in an Oracle database.

Software Required

- Oracle Database 11g / 12c / 19c
- SQL*Plus or Oracle SQL Developer

Theory

Normalization is the process of organizing data in a database so as to reduce redundancy and improve data integrity. Its objectives include eliminating data redundancy, avoiding insertion, deletion, and update anomalies, ensuring data consistency, and improving overall database efficiency.

A Functional Dependency (FD) exists when one attribute uniquely determines another. The notation $A \rightarrow B$ means that attribute A determines attribute B — for example, $STUDENT_ID \rightarrow STUDENT_NAME$, $DEPT_ID \rightarrow DEPT_NAME$.

Procedure

1. Examine the unnormalized student-course data.
2. Convert the data to First Normal Form (1NF) by removing repeating groups.
3. Convert the data to Second Normal Form (2NF) by removing partial dependencies.
4. Convert the data to Third Normal Form (3NF) by removing transitive dependencies.
5. Convert the data to Fourth Normal Form (4NF) by removing multivalued dependencies.
6. Convert the data to Fifth Normal Form (5NF) by removing join dependencies.
7. Create the corresponding Oracle tables at each stage.

Program

Unnormalized Form (UNF)

Consider the following STUDENT_COURSE data, which contains repeating groups, data redundancy, and is difficult to update.

Student_ID	Student_Name	Course1	Course2	Course3
101	ARUN	DBMS	JAVA	C
102	BALA	DBMS	C++	NULL
103	DIVYA	JAVA	PYTHON	DBMS

First Normal Form (1NF)

Rule: eliminate repeating groups, ensure each column holds atomic values, and ensure each row is unique.

Student_ID	Student_Name	Course
101	ARUN	DBMS
101	ARUN	JAVA
101	ARUN	C
102	BALA	DBMS
102	BALA	C++
103	DIVYA	JAVA
103	DIVYA	PYTHON
103	DIVYA	DBMS

Functional dependency: (Student_ID, Course) → Student_Name

```
CREATE TABLE STUDENT_COURSE_1NF
(
  STUDENT_ID NUMBER(5),
  STUDENT_NAME VARCHAR2(30),
  COURSE VARCHAR2(20)
);
```

Output

Table created.

Second Normal Form (2NF)

Rule: the table must already be in 1NF, and all partial dependencies must be removed so that non-key attributes depend on the entire primary key. Here, the composite key is (Student_ID, Course), and Student_ID → Student_Name is a partial dependency.

Decomposition – STUDENT table:

Student_ID	Student_Name
101	ARUN
102	BALA
103	DIVYA

Decomposition – STUDENT_COURSE table:

Student_ID	Course
101	DBMS
101	JAVA
101	C

Student_ID	Course
102	DBMS
102	C++
103	JAVA

```

CREATE TABLE STUDENT
(
    STUDENT_ID NUMBER(5) PRIMARY KEY,
    STUDENT_NAME VARCHAR2(30)
);

```

Output

Table created.

```

CREATE TABLE STUDENT_COURSE
(
    STUDENT_ID NUMBER(5),
    COURSE VARCHAR2(20),
    PRIMARY KEY (STUDENT_ID, COURSE),
    FOREIGN KEY (STUDENT_ID)
        REFERENCES STUDENT(STUDENT_ID)
);

```

Output

Table created.

Third Normal Form (3NF)

Rule: the table must already be in 2NF, and all transitive dependencies must be removed. Here, $STUDENT_ID \rightarrow DEPT_ID$ and $DEPT_ID \rightarrow DEPT_NAME$ together create the transitive dependency $STUDENT_ID \rightarrow DEPT_NAME$.

Decomposition – STUDENT table:

Student_ID	Student_Name	Dept_ID
101	ARUN	D01
102	BALA	D02
103	DIVYA	D01

Decomposition – DEPARTMENT table:

Dept_ID	Dept_Name
D01	CSE
D02	ECE

```

CREATE TABLE DEPARTMENT

```

Dept_ID	Dept_Name
<pre>(DEPT_ID VARCHAR2(5) PRIMARY KEY, DEPT_NAME VARCHAR2(30));</pre>	

Output

Table created.

<pre>CREATE TABLE STUDENT (STUDENT_ID NUMBER(5) PRIMARY KEY, STUDENT_NAME VARCHAR2(30), DEPT_ID VARCHAR2(5), FOREIGN KEY (DEPT_ID) REFERENCES DEPARTMENT(DEPT_ID));</pre>

Output

Table created.

Fourth Normal Form (4NF)

Rule: the table must already be in BCNF, and all multivalued dependencies must be removed. For example, a student may have several hobbies and several languages, giving Student $\twoheadrightarrow$ Hobby and Student $\twoheadrightarrow$ Language.

Original STUDENT_DETAILS:

Student	Hobby	Language
Arun	Cricket	Tamil
Arun	Music	English
Bala	Reading	Tamil

Decomposition – STUDENT_HOBBY and STUDENT_LANGUAGE:

Student	Hobby
Arun	Cricket
Arun	Music
Student	Language
Arun	Tamil
Arun	English
<pre>CREATE TABLE STUDENT_HOBBY (</pre>	

Student	Hobby
<pre> STUDENT_NAME VARCHAR2(30), HOBBY VARCHAR2(30)); CREATE TABLE STUDENT_LANGUAGE (STUDENT_NAME VARCHAR2(30), LANGUAGE VARCHAR2(30)); </pre>	

Output

Table created.

Table created.

Fifth Normal Form (5NF)

Rule: the table must already be in 4NF, and all join dependencies must be removed. Consider the following Supplier-Part-Project relationship.

Supplier	Part	Project
S1	P1	J1
S1	P2	J1
S2	P1	J2

This table can be losslessly decomposed into three pairwise relations:

Supplier	Part
S1	P1
S1	P2
S2	P1
Part	Project
P1	J1
P2	J1
P1	J2
Supplier	Project
S1	J1
S2	J2
<pre> CREATE TABLE SUPPLIER_PART (</pre>	

Supplier	Part
<pre> SUPPLIER_ID VARCHAR2(5), PART_ID VARCHAR2(5)); CREATE TABLE PART_PROJECT (PART_ID VARCHAR2(5), PROJECT_ID VARCHAR2(5)); CREATE TABLE SUPPLIER_PROJECT (SUPPLIER_ID VARCHAR2(5), PROJECT_ID VARCHAR2(5)); </pre>	

Output

Table created.

Table created.

Table created.

Summary of Normal Forms

Normal Form	Removes
1NF	Repeating groups
2NF	Partial dependency
3NF	Transitive dependency
4NF	Multivalued dependency
5NF	Join dependency

Viva Voce Questions

1. What is normalization?
2. Why is normalization required?
3. Define Functional Dependency.
4. Explain 1NF with an example.
5. What is a partial dependency?
6. What is a transitive dependency?
7. Define multivalued dependency.
8. What is a join dependency?
9. Differentiate between 3NF and 4NF.

10. What are the advantages of normalization?

Result

Thus, normalization up to the Fifth Normal Form (1NF, 2NF, 3NF, 4NF, and 5NF) was studied and implemented using functional dependencies in an Oracle database successfully.

Experiment No. 9

Creating Relationships Between Tables and Performing Joins and Subqueries in Oracle

Aim

To create relationships between tables using Primary Key and Foreign Key constraints, and to perform various types of joins and subqueries in an Oracle database.

Software Required

- Oracle Database 11g / 12c / 19c
- SQL*Plus or Oracle SQL Developer

Theory

A relationship between tables is established using a Primary Key in the parent table and a matching Foreign Key in the child table.

Oracle supports several types of joins:

- Inner Join
- Left Outer Join
- Right Outer Join
- Full Outer Join
- Self Join

A subquery is a query nested inside another query, used either to filter results or to compute a value used by the outer query.

Procedure

1. Create the DEPARTMENT table.
2. Create the STUDENT table with a foreign key.
3. Insert records into both tables.
4. Perform Inner, Left Outer, Right Outer, Full Outer, and Self Joins.
5. Perform subqueries, subqueries with IN, and correlated subqueries.
6. Verify the outputs.

Program

Step 1: Create the DEPARTMENT Table

```
CREATE TABLE DEPARTMENT
(
    DEPT_ID NUMBER(3) PRIMARY KEY,
    DEPT_NAME VARCHAR2(30)
);
```

Output

Table created.

Step 2: Insert Records into DEPARTMENT

```
INSERT INTO DEPARTMENT VALUES (101, 'CSE');  
INSERT INTO DEPARTMENT VALUES (102, 'ECE');  
INSERT INTO DEPARTMENT VALUES (103, 'IT');
```

Output

1 row created.
1 row created.
1 row created.

Step 3: Create the STUDENT Table

```
CREATE TABLE STUDENT  
(  
    STUDENT_ID NUMBER(5) PRIMARY KEY,  
    STUDENT_NAME VARCHAR2(30),  
    MARKS NUMBER(3),  
    DEPT_ID NUMBER(3),  
  
    CONSTRAINT FK_DEPT FOREIGN KEY (DEPT_ID)  
        REFERENCES DEPARTMENT(DEPT_ID)  
);
```

Output

Table created.

Step 4: Insert Records into STUDENT

```
INSERT INTO STUDENT VALUES (1001, 'ARUN', 85, 101);  
INSERT INTO STUDENT VALUES (1002, 'BALA', 78, 102);  
INSERT INTO STUDENT VALUES (1003, 'DIVYA', 92, 103);  
INSERT INTO STUDENT VALUES (1004, 'KAVI', 88, 101);
```

Output

1 row created.
1 row created.
1 row created.
1 row created.

INNER JOIN – Display Student and Department Details

```
SELECT S.STUDENT_ID, S.STUDENT_NAME, D.DEPT_NAME  
FROM STUDENT S
```

```
INNER JOIN DEPARTMENT D
  ON S.DEPT_ID = D.DEPT_ID;
```

Output

STUDENT_ID	STUDENT_NAME	DEPT_NAME
-----	-----	-----
1001	ARUN	CSE
1002	BALA	ECE
1003	DIVYA	IT
1004	KAVI	CSE

LEFT OUTER JOIN – All Students with Department Information

```
SELECT S.STUDENT_NAME, D.DEPT_NAME
FROM STUDENT S
LEFT OUTER JOIN DEPARTMENT D
  ON S.DEPT_ID = D.DEPT_ID;
```

Output

STUDENT_NAME	DEPT_NAME
-----	-----
ARUN	CSE
BALA	ECE
DIVYA	IT
KAVI	CSE

RIGHT OUTER JOIN – All Departments and Students

```
SELECT S.STUDENT_NAME, D.DEPT_NAME
FROM STUDENT S
RIGHT OUTER JOIN DEPARTMENT D
  ON S.DEPT_ID = D.DEPT_ID;
```

Output

STUDENT_NAME	DEPT_NAME
-----	-----
ARUN	CSE
KAVI	CSE
BALA	ECE
DIVYA	IT

FULL OUTER JOIN

```
SELECT S.STUDENT_NAME, D.DEPT_NAME
FROM STUDENT S
FULL OUTER JOIN DEPARTMENT D
  ON S.DEPT_ID = D.DEPT_ID;
```

Output

STUDENT_NAME	DEPT_NAME
-----	-----
ARUN	CSE
KAVI	CSE
BALA	ECE
DIVYA	IT

SELF JOIN – Create the EMPLOYEE Table

```
CREATE TABLE EMPLOYEE
(
    EMP_ID NUMBER(5) PRIMARY KEY,
    EMP_NAME VARCHAR2(30),
    MANAGER_ID NUMBER(5)
);

INSERT INTO EMPLOYEE VALUES (1, 'RAJ', NULL);
INSERT INTO EMPLOYEE VALUES (2, 'ARUN', 1);
INSERT INTO EMPLOYEE VALUES (3, 'BALA', 1);
```

Self join query:

```
SELECT E.EMP_NAME EMPLOYEE, M.EMP_NAME MANAGER
FROM EMPLOYEE E
LEFT JOIN EMPLOYEE M
    ON E.MANAGER_ID = M.EMP_ID;
```

Output

EMPLOYEE	MANAGER
-----	-----
RAJ	
ARUN	RAJ
BALA	RAJ

SUBQUERY – Students Scoring Above Average Marks

```
SELECT STUDENT_NAME, MARKS
FROM STUDENT
WHERE MARKS >
(
    SELECT AVG(MARKS)
    FROM STUDENT
);
```

Output

STUDENT_NAME	MARKS
-----	-----
DIVYA	92
KAVI	88

Average marks = 85.75

SUBQUERY WITH IN – Students in the CSE Department

```
SELECT STUDENT_NAME
FROM STUDENT
WHERE DEPT_ID IN
(
    SELECT DEPT_ID
    FROM DEPARTMENT
    WHERE DEPT_NAME = 'CSE'
);
```

Output

```
STUDENT_NAME
-----
ARUN
KAVI
```

CORRELATED SUBQUERY – Students with Maximum Marks in Their Department

```
SELECT STUDENT_NAME, MARKS, DEPT_ID
FROM STUDENT S1
WHERE MARKS =
(
    SELECT MAX(MARKS)
    FROM STUDENT S2
    WHERE S1.DEPT_ID = S2.DEPT_ID
);
```

Output

STUDENT_NAME	MARKS	DEPT_ID
-----	-----	-----
KAVI	88	101
BALA	78	102
DIVYA	92	103

Viva Voce Questions

1. What is a Primary Key?
2. What is a Foreign Key?
3. Why are relationships used in databases?
4. What is an Inner Join?
5. Differentiate between an Inner Join and an Outer Join.
6. What is a Self Join?
7. What is a Subquery?
8. What is a Correlated Subquery?
9. Can a subquery return multiple rows?
10. What are the advantages of joins?

Result

Thus, relationships between tables were created successfully using Primary Key and Foreign Key constraints, and various joins and subqueries were executed successfully in an Oracle database.

Experiment No. 10 (a)

PL/SQL Block to Satisfy Conditions by Accepting Input from the User

Aim

To write a PL/SQL block that accepts a number from the user and checks whether it is positive, negative, or zero.

Software Required

- Oracle Database 11g / 12c / 19c
- SQL*Plus or Oracle SQL Developer

Theory

PL/SQL (Procedural Language / Structured Query Language) is Oracle's procedural extension of SQL. It supports variables, conditional statements, loops, exception handling, procedures, and functions. The IF-THEN-ELSE statement is used to execute a block of statements based on a specified condition.

```
IF condition THEN
    statements;
ELSIF condition THEN
    statements;
ELSE
    statements;
END IF;
```

Procedure

1. Enable server output.
2. Accept a number from the user.
3. Check whether the number is positive, negative, or zero.
4. Display the appropriate message.

Program

PL/SQL Block

```
SET SERVEROUTPUT ON;

DECLARE
    N NUMBER;
BEGIN
    N := &N;

    IF N > 0 THEN
        DBMS_OUTPUT.PUT_LINE('The number is Positive');
```

```
ELSIF N < 0 THEN
    DBMS_OUTPUT.PUT_LINE('The number is Negative');
ELSE
    DBMS_OUTPUT.PUT_LINE('The number is Zero');
END IF;
END;
/
```

Sample Run 1 – Enter value for N: 25

Output

```
old   5: N := &N;
new   5: N := 25;

The number is Positive

PL/SQL procedure successfully completed.
```

Sample Run 2 – Enter value for N: -15

Output

```
old   5: N := &N;
new   5: N := -15;

The number is Negative

PL/SQL procedure successfully completed.
```

Sample Run 3 – Enter value for N: 0

Output

```
old   5: N := &N;
new   5: N := 0;

The number is Zero

PL/SQL procedure successfully completed.
```

Viva Voce Questions

1. What is PL/SQL?
2. What is the purpose of SET SERVEROUTPUT ON?
3. What is DBMS_OUTPUT.PUT_LINE?
4. What is a substitution variable?
5. Explain the IF-ELSIF-ELSE statement.
6. What is the difference between SQL and PL/SQL?
7. Why is a forward slash (/) used after the END statement?

8. Can nested IF statements be used in PL/SQL?
9. What are the advantages of PL/SQL?
10. What is a PL/SQL block?

Result

Thus, a PL/SQL block was written successfully to accept input from the user and determine whether the entered number is positive, negative, or zero.

Experiment No. 10 (b)

PL/SQL Block to Handle Exceptions in Oracle

Aim

To write PL/SQL blocks that handle various predefined exceptions using the EXCEPTION handling mechanism.

Software Required

- Oracle Database 11g / 12c / 19c
- SQL*Plus or Oracle SQL Developer

Theory

Exception handling in PL/SQL is used to handle runtime errors gracefully. When an error occurs, control is transferred to the EXCEPTION section of the PL/SQL block. Exceptions may be predefined (such as NO_DATA_FOUND, TOO_MANY_ROWS, ZERO_DIVIDE, VALUE_ERROR, and DUP_VAL_ON_INDEX), non-predefined, or user-defined.

Procedure

1. Enable server output.
2. Create a sample table.
3. Execute PL/SQL blocks that generate exceptions.
4. Handle each exception using the EXCEPTION block.
5. Display appropriate messages.

Program

Program 1: Handling the ZERO_DIVIDE Exception

```
SET SERVEROUTPUT ON;

DECLARE
    A NUMBER := 100;
    B NUMBER := 0;
    C NUMBER;
BEGIN
    C := A / B;
    DBMS_OUTPUT.PUT_LINE('Result = ' || C);
EXCEPTION
    WHEN ZERO_DIVIDE THEN
        DBMS_OUTPUT.PUT_LINE('Error: Division by Zero is not allowed.');
```

```
END;
/
```

Output

Error: Division by Zero is not allowed.

PL/SQL procedure successfully completed.

Program 2: Handling the NO_DATA_FOUND Exception

Create a sample table:

```
CREATE TABLE STUDENT
(
    STUDENT_ID NUMBER PRIMARY KEY,
    STUDENT_NAME VARCHAR2(30)
);

INSERT INTO STUDENT VALUES (101,'ARUN');
COMMIT;

SET SERVEROUTPUT ON;

DECLARE
    V_NAME STUDENT.STUDENT_NAME%TYPE;
BEGIN
    SELECT STUDENT_NAME
    INTO V_NAME
    FROM STUDENT
    WHERE STUDENT_ID = 999;

    DBMS_OUTPUT.PUT_LINE(V_NAME);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Error: No matching record found.');
```

END;
/

Output

Error: No matching record found.

PL/SQL procedure successfully completed.

Program 3: Handling the TOO_MANY_ROWS Exception

```
SET SERVEROUTPUT ON;

DECLARE
    V_NAME STUDENT.STUDENT_NAME%TYPE;
BEGIN
    SELECT STUDENT_NAME
    INTO V_NAME
    FROM STUDENT;
```

```
        DBMS_OUTPUT.PUT_LINE(V_NAME);
    EXCEPTION
        WHEN TOO_MANY_ROWS THEN
            DBMS_OUTPUT.PUT_LINE('Error: Query returned multiple rows.');
```

```
END;
/
```

Output

```
Error: Query returned multiple rows.

PL/SQL procedure successfully completed.
```

Program 4: Handling the DUP_VAL_ON_INDEX Exception

```
SET SERVEROUTPUT ON;

BEGIN
    INSERT INTO STUDENT VALUES (101,'BALA');
EXCEPTION
    WHEN DUP_VAL_ON_INDEX THEN
        DBMS_OUTPUT.PUT_LINE('Error: Duplicate value violates unique constraint.');
```

```
END;
/
```

Output

```
Error: Duplicate value violates unique constraint.

PL/SQL procedure successfully completed.
```

Program 5: Handling the VALUE_ERROR Exception

```
SET SERVEROUTPUT ON;

DECLARE
    V_NUM NUMBER(2);
BEGIN
    V_NUM := 12345;
EXCEPTION
    WHEN VALUE_ERROR THEN
        DBMS_OUTPUT.PUT_LINE('Error: Value exceeds variable size.');
```

```
END;
/
```

Output

```
Error: Value exceeds variable size.

PL/SQL procedure successfully completed.
```

Program 6: Using the OTHERS Handler

```
SET SERVEROUTPUT ON;

DECLARE
    V_NUM NUMBER;
BEGIN
    V_NUM := 10/0;
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('SQLCODE : ' || SQLCODE);
        DBMS_OUTPUT.PUT_LINE('SQLERRM : ' || SQLERRM);
END;
/
```

Output

```
SQLCODE : -1476
SQLERRM : ORA-01476: divisor is equal to zero

PL/SQL procedure successfully completed.
```

Summary of Exceptions

Exception	Description
ZERO_DIVIDE	Division by zero
NO_DATA_FOUND	Query returns no rows
TOO_MANY_ROWS	Query returns multiple rows
DUP_VAL_ON_INDEX	Duplicate value in a unique index
VALUE_ERROR	Arithmetic or conversion error
OTHERS	Handles all remaining, unhandled exceptions

Viva Voce Questions

1. What is exception handling in PL/SQL?
2. What are predefined exceptions?
3. Explain the ZERO_DIVIDE exception.
4. When does NO_DATA_FOUND occur?
5. What causes the TOO_MANY_ROWS exception?
6. What is DUP_VAL_ON_INDEX?
7. Explain VALUE_ERROR.
8. What is the purpose of the OTHERS handler?
9. What are SQLCODE and SQLERRM?
10. What are user-defined exceptions?

Result

Thus, PL/SQL programs were written successfully to handle predefined exceptions such as ZERO_DIVIDE, NO_DATA_FOUND, TOO_MANY_ROWS, DUP_VAL_ON_INDEX, VALUE_ERROR, and OTHERS in an Oracle database.

Experiment No. 10 (c)

Creation of Procedures and Functions in Oracle PL/SQL

Aim

To create and execute procedures and functions using PL/SQL in an Oracle database.

Software Required

- Oracle Database 11g / 12c / 19c
- SQL*Plus or Oracle SQL Developer

Theory

A Procedure is a named PL/SQL block that performs a specific task; it may accept parameters and can return multiple values through OUT parameters. A Function is a named PL/SQL block that returns exactly one value using the RETURN statement.

Procedure	Function
Performs an action	Returns a value
May or may not return a value	Must return exactly one value
Called using EXECUTE or another block	Can be used directly inside SQL statements
Uses IN, OUT, and IN OUT parameters	Generally uses only IN parameters

Procedure

1. Create a STUDENT table.
2. Insert sample records.
3. Create a procedure to display student details.
4. Execute the procedure.
5. Create a function to calculate the grade.
6. Execute the function and verify the output.

Program

Step 1: Create the STUDENT Table

```
CREATE TABLE STUDENT
(
  STUDENT_ID NUMBER(5) PRIMARY KEY,
  STUDENT_NAME VARCHAR2(30),
  MARKS NUMBER(3)
);
```

Output

Table created.

Step 2: Insert Sample Records

```
INSERT INTO STUDENT VALUES (101,'ARUN',85);
INSERT INTO STUDENT VALUES (102,'BALA',72);
INSERT INTO STUDENT VALUES (103,'DIVYA',91);
COMMIT;
```

Output

```
1 row created.
1 row created.
1 row created.
Commit complete.
```

Step 3: Create a Procedure to Display Student Details

```
CREATE OR REPLACE PROCEDURE GET_STUDENT
(
    P_ID IN NUMBER
)
IS
    V_NAME STUDENT.STUDENT_NAME%TYPE;
    V_MARKS STUDENT.MARKS%TYPE;
BEGIN
    SELECT STUDENT_NAME, MARKS
    INTO V_NAME, V_MARKS
    FROM STUDENT
    WHERE STUDENT_ID = P_ID;

    DBMS_OUTPUT.PUT_LINE('Student Name : ' || V_NAME);
    DBMS_OUTPUT.PUT_LINE('Marks          : ' || V_MARKS);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Student Record Not Found.');
```

```
END;
/
```

Output

Procedure created.

Step 4: Execute the Procedure

```
SET SERVEROUTPUT ON;

EXEC GET_STUDENT(101);
```

Output

Student Name : ARUN
Marks : 85

PL/SQL procedure successfully completed.

Sample output for an invalid input:

```
EXEC GET_STUDENT(110);
```

Output

Student Record Not Found.
PL/SQL procedure successfully completed.

Step 5: Create a Function to Find the Grade

Marks	Grade
90 and above	O
80–89	A+
70–79	A
60–69	B
Below 60	C

```
CREATE OR REPLACE FUNCTION FIND_GRADE
(
    P_MARKS NUMBER
)
RETURN VARCHAR2
IS
    V_GRADE VARCHAR2(2);
BEGIN
    IF P_MARKS >= 90 THEN
        V_GRADE := 'O';
    ELSIF P_MARKS >= 80 THEN
        V_GRADE := 'A+';
    ELSIF P_MARKS >= 70 THEN
        V_GRADE := 'A';
    ELSIF P_MARKS >= 60 THEN
        V_GRADE := 'B';
    ELSE
        V_GRADE := 'C';
    END IF;

    RETURN V_GRADE;
END;
/
```

Output

Function created.

Step 6: Execute the Function

```
SELECT FIND_GRADE(91) AS GRADE FROM DUAL;
```

Output

```
GRADE
-----
0
```

```
SELECT FIND_GRADE(75) AS GRADE FROM DUAL;
```

Output

```
GRADE
-----
A
```

Step 7: Use the Function with Table Data

```
SELECT STUDENT_ID, STUDENT_NAME, MARKS,
       FIND_GRADE(MARKS) AS GRADE
FROM STUDENT;
```

Output

STUDENT_ID	STUDENT_NAME	MARKS	GRADE
-----	-----	-----	-----
101	ARUN	85	A+
102	BALA	72	A
103	DIVYA	91	O

Verify the Created Objects

```
SELECT OBJECT_NAME, OBJECT_TYPE
FROM USER_OBJECTS
WHERE OBJECT_TYPE = 'PROCEDURE';
```

Output

OBJECT_NAME	OBJECT_TYPE
-----	-----
GET_STUDENT	PROCEDURE

```
SELECT OBJECT_NAME, OBJECT_TYPE
FROM USER_OBJECTS
WHERE OBJECT_TYPE = 'FUNCTION';
```

Output

OBJECT_NAME	OBJECT_TYPE
-----	-----
FIND_GRADE	FUNCTION

Viva Voce Questions

1. What is a Procedure in PL/SQL?
2. What is a Function?
3. What is the difference between a Procedure and a Function?
4. What are IN, OUT, and IN OUT parameters?
5. Can a Function return multiple values?
6. Can a Function be used inside a SELECT statement?
7. What is the purpose of the RETURN statement?
8. What is CREATE OR REPLACE?
9. What exception occurs if a SELECT INTO statement returns no rows?
10. How do you execute a Procedure in Oracle?

Result

Thus, procedures and functions were created and executed successfully in Oracle PL/SQL, and the outputs were verified.

Experiment No. 10 (d)

Creation of Database Triggers (Insert Trigger, Delete Trigger, Update Trigger)

Aim

To create and execute database triggers for INSERT, DELETE, and UPDATE operations in an Oracle database.

Software Required

- Oracle Database 11g / 12c / 19c
- SQL*Plus or Oracle SQL Developer

Theory

A Trigger is a stored PL/SQL block that executes automatically in response to a specified event occurring on a table or view, such as an INSERT, UPDATE, or DELETE. Triggers can additionally be classified as BEFORE or AFTER, and as row-level or statement-level.

```
CREATE OR REPLACE TRIGGER trigger_name
BEFORE|AFTER INSERT|UPDATE|DELETE
ON table_name
FOR EACH ROW
BEGIN
    statements;
END;
/
```

Procedure

1. Create the EMPLOYEE and EMP_AUDIT tables.
2. Insert sample records into EMPLOYEE.
3. Create an INSERT trigger.
4. Create an UPDATE trigger.
5. Create a DELETE trigger.
6. Execute DML operations.
7. Verify the audit table.

Program

Step 1: Create the EMPLOYEE Table

```
CREATE TABLE EMPLOYEE
(
    EMP_ID NUMBER(5) PRIMARY KEY,
    EMP_NAME VARCHAR2(30),
    SALARY NUMBER(8)
```

```
);
```

Output

```
Table created.
```

Step 2: Create the EMP_AUDIT Table

```
CREATE TABLE EMP_AUDIT
(
    AUDIT_ID NUMBER(5),
    EMP_ID NUMBER(5),
    ACTION_PERFORMED VARCHAR2(20),
    ACTION_DATE DATE
);
```

Output

```
Table created.
```

Step 3: Insert Sample Records

```
INSERT INTO EMPLOYEE VALUES (101, 'ARUN', 35000);
INSERT INTO EMPLOYEE VALUES (102, 'BALA', 40000);
COMMIT;
```

Output

```
1 row created.
1 row created.
Commit complete.
```

INSERT Trigger – Step 4: Create the Trigger

```
CREATE OR REPLACE TRIGGER TRG_EMP_INSERT
AFTER INSERT
ON EMPLOYEE
FOR EACH ROW
BEGIN
    INSERT INTO EMP_AUDIT
    VALUES (EMP_AUDIT_SEQ.NEXTVAL, :NEW.EMP_ID, 'INSERT', SYSDATE);
END;
/
```

Output

```
Trigger created.
```

Create the supporting sequence for the audit ID:

```
CREATE SEQUENCE EMP_AUDIT_SEQ
START WITH 1
```

```
INCREMENT BY 1;
```

Output

```
Sequence created.
```

Test the INSERT trigger:

```
INSERT INTO EMPLOYEE VALUES (103, 'DIVYA', 45000);
```

Output

```
1 row created.
```

Verify the audit table:

```
SELECT * FROM EMP_AUDIT;
```

Output

AUDIT_ID	EMP_ID	ACTION_PERFORMED	ACTION_DATE
-----	-----	-----	-----
1	103	INSERT	25-JUL-26

UPDATE Trigger – Step 5: Create the Trigger

```
CREATE OR REPLACE TRIGGER TRG_EMP_UPDATE
AFTER UPDATE
ON EMPLOYEE
FOR EACH ROW
BEGIN
    INSERT INTO EMP_AUDIT
    VALUES (EMP_AUDIT_SEQ.NEXTVAL, :NEW.EMP_ID, 'UPDATE', SYSDATE);
END;
/
```

Output

```
Trigger created.
```

Test the UPDATE trigger:

```
UPDATE EMPLOYEE
SET SALARY = 50000
WHERE EMP_ID = 101;
```

Output

```
1 row updated.
```

Verify the audit table:

```
SELECT * FROM EMP_AUDIT;
```

Output

AUDIT_ID	EMP_ID	ACTION_PERFORMED	ACTION_DATE
-----	-----	-----	-----
1	103	INSERT	25-JUL-26
2	101	UPDATE	25-JUL-26

DELETE Trigger – Step 6: Create the Trigger

```
CREATE OR REPLACE TRIGGER TRG_EMP_DELETE
AFTER DELETE
ON EMPLOYEE
FOR EACH ROW
BEGIN
    INSERT INTO EMP_AUDIT
    VALUES (EMP_AUDIT_SEQ.NEXTVAL, :OLD.EMP_ID, 'DELETE', SYSDATE);
END;
/
```

Output

Trigger created.

Test the DELETE trigger:

```
DELETE FROM EMPLOYEE
WHERE EMP_ID = 102;
```

Output

1 row deleted.

Verify the audit table:

```
SELECT * FROM EMP_AUDIT;
```

Output

AUDIT_ID	EMP_ID	ACTION_PERFORMED	ACTION_DATE
-----	-----	-----	-----
1	103	INSERT	25-JUL-26
2	101	UPDATE	25-JUL-26
3	102	DELETE	25-JUL-26

Display the EMPLOYEE Table

```
SELECT * FROM EMPLOYEE;
```

Output

EMP_ID	EMP_NAME	SALARY
-----	-----	-----
101	ARUN	50000

103	DIVYA	45000
-----	-------	-------

Explanation of :NEW and :OLD

Pseudorecord	Purpose
:NEW	Refers to the new values during INSERT and UPDATE operations
:OLD	Refers to the old values during DELETE and UPDATE operations

Viva Voce Questions

1. What is a Trigger?
2. What are the different types of triggers?
3. What is the difference between BEFORE and AFTER triggers?
4. What is a Row-Level Trigger?
5. What is the purpose of FOR EACH ROW?
6. Explain the use of :NEW.
7. Explain the use of :OLD.
8. Can multiple triggers be created on the same table?
9. What is an Audit Table?
10. What are the advantages of triggers?

Result

Thus, the INSERT, UPDATE, and DELETE triggers were created successfully in an Oracle database, and the corresponding actions were recorded automatically in the audit table.

Experiment No. 10 (e)

Creation of Database Operations with Implicit and Explicit Cursors in Oracle

Aim

To write PL/SQL programs using an Implicit Cursor and an Explicit Cursor in an Oracle database.

Software Required

- Oracle Database 11g / 12c / 19c
- SQL*Plus or Oracle SQL Developer

Theory

A cursor is a pointer to the context area that stores the result set of an SQL statement. An Implicit Cursor is created automatically by Oracle for DML statements such as INSERT, UPDATE, DELETE, and SELECT INTO, with Oracle handling its opening, fetching, and closing internally. An Explicit Cursor, by contrast, is declared by the programmer and is used when a query is expected to return multiple rows; the programmer must explicitly open, fetch from, and close it.

Procedure

1. Create the EMPLOYEE table.
2. Insert sample records.
3. Write a PL/SQL block using an implicit cursor.
4. Write a PL/SQL block using an explicit cursor.
5. Execute the programs and verify the output.

Program

Step 1: Create the EMPLOYEE Table

```
CREATE TABLE EMPLOYEE
(
    EMP_ID NUMBER(5) PRIMARY KEY,
    EMP_NAME VARCHAR2(30),
    DEPARTMENT VARCHAR2(20),
    SALARY NUMBER(8)
);
```

Output

Table created.

Step 2: Insert Sample Records

```
INSERT INTO EMPLOYEE VALUES (101, 'ARUN', 'CSE', 35000);
```

```

INSERT INTO EMPLOYEE VALUES (102, 'BALA', 'ECE', 32000);
INSERT INTO EMPLOYEE VALUES (103, 'DIVYA', 'IT', 45000);
INSERT INTO EMPLOYEE VALUES (104, 'KAVI', 'CSE', 40000);
COMMIT;

```

Output

```

1 row created.
1 row created.
1 row created.
1 row created.
Commit complete.

```

Part A – Implicit Cursor

Aim: to use an implicit cursor to update employee salaries and display the number of rows affected.

```

SET SERVEROUTPUT ON;

BEGIN
    UPDATE EMPLOYEE
    SET SALARY = SALARY + 5000
    WHERE DEPARTMENT = 'CSE';

    DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT || ' row(s) updated. ');
    COMMIT;
END;
/

```

Output

```

2 row(s) updated.

PL/SQL procedure successfully completed.

```

Verify the update:

```

SELECT * FROM EMPLOYEE;

```

Output

EMP_ID	EMP_NAME	DEPARTMENT	SALARY
101	ARUN	CSE	40000
102	BALA	ECE	32000
103	DIVYA	IT	45000
104	KAVI	CSE	45000

Attribute	Description
SQL%FOUND	Returns TRUE if rows were affected
SQL%NOTFOUND	Returns TRUE if no rows were affected

SQL%ROWCOUNT	Returns the number of rows affected
SQL%ISOPEN	Always returns FALSE for implicit cursors

Part B – Explicit Cursor

Aim: to use an explicit cursor to retrieve and display multiple employee records.

```

SET SERVEROUTPUT ON;

DECLARE
    CURSOR EMP_CURSOR IS
        SELECT EMP_ID, EMP_NAME, DEPARTMENT, SALARY
        FROM EMPLOYEE;

    V_ID EMPLOYEE.EMP_ID%TYPE;
    V_NAME EMPLOYEE.EMP_NAME%TYPE;
    V_DEPT EMPLOYEE.DEPARTMENT%TYPE;
    V_SAL EMPLOYEE.SALARY%TYPE;
BEGIN
    OPEN EMP_CURSOR;
    LOOP
        FETCH EMP_CURSOR INTO V_ID, V_NAME, V_DEPT, V_SAL;
        EXIT WHEN EMP_CURSOR%NOTFOUND;

        DBMS_OUTPUT.PUT_LINE(V_ID || ' ' || V_NAME || ' ' ||
                               V_DEPT || ' ' || V_SAL);
    END LOOP;
    CLOSE EMP_CURSOR;
END;
/

```

Output

```

101  ARUN   CSE    40000
102  BALA   ECE    32000
103  DIVYA  IT     45000
104  KAVI   CSE    45000

```

PL/SQL procedure successfully completed.

Attribute	Description
cursor_name%FOUND	TRUE if the last fetch returned a row
cursor_name%NOTFOUND	TRUE if the last fetch failed to return a row
cursor_name%ROWCOUNT	Number of rows fetched so far
cursor_name%ISOPEN	TRUE if the cursor is currently open

Using a Cursor FOR Loop

```

SET SERVEROUTPUT ON;

DECLARE
    CURSOR EMP_CURSOR IS
        SELECT EMP_ID, EMP_NAME, SALARY
        FROM EMPLOYEE;
BEGIN
    FOR EMP_REC IN EMP_CURSOR LOOP
        DBMS_OUTPUT.PUT_LINE(EMP_REC.EMP_ID || ' ' ||
                               EMP_REC.EMP_NAME || ' ' ||
                               EMP_REC.SALARY);
    END LOOP;
END;
/

```

Output

```

101 ARUN 40000
102 BALA 32000
103 DIVYA 45000
104 KAVI 45000

```

PL/SQL procedure successfully completed.

Implicit Cursor vs. Explicit Cursor

Implicit Cursor	Explicit Cursor
Created automatically by Oracle	Created explicitly by the programmer
Used for single-row processing	Used for multiple-row processing
No OPEN, FETCH, or CLOSE required	Requires OPEN, FETCH, and CLOSE
Simpler to use	Provides more control over processing
Uses SQL cursor attributes	Uses cursor-name attributes

Viva Voce Questions

1. What is a cursor?
2. What is an implicit cursor?
3. What is an explicit cursor?
4. What are cursor attributes?
5. Explain SQL%ROWCOUNT.
6. Why are explicit cursors used?
7. What is the purpose of FETCH?
8. What happens if an explicit cursor is not closed?
9. What is a Cursor FOR Loop?
10. Differentiate between implicit and explicit cursors.

Result

Thus, PL/SQL programs using an implicit cursor and an explicit cursor were written and executed successfully in an Oracle database, and the outputs were verified.